

Module 5: Some Important Cyclic Codes & Convolution Codes.

Convolutional codes.

The main difference between block codes and convolutional codes is that, in "block codes", a block of " n "-digits generated by an encoder in a particular time-unit depends only on the block of ' k ' input message digits within that time unit. In convolutional codes, a block of ' n ' code digits generated by the encoder in a time unit depends not only on the block of ' k ' message digits within that time unit, but also on the preceding $(m-1)$ blocks of message digits ($m > 1$).

Like block codes, convolutional codes can be designed to either detect or correct the errors. Block codes are better suited for error detections & convolutional codes for error correction.

Encoding of convolutional codes can be accomplished using simple shift register. and Decoding is done using several practical procedure.

Encoders for Convolutional codes:

A convolutional encoder, takes a sequence of message digits and generates sequence of code digits.

In any time unit, a message block consisting of 'k' digits is fed into the encoder & the encoder generates a code block consisting of 'n' code digits ($k \leq n$). The n-digit code block depends not only on the k-digit message block of the same time, but also on the previous (m-1) message blocks.

The code generated by the encoder is called as (n, k, m) convolutional code of constraint length nm digits & rate efficiency k/n .

The parameters of the encoder n, k and m are in general small integers and encoder consists of shift register, modulo 2 adders.

(P1) An encoder for $(n, k, m) = (3, 1, 3)$ convolutional code is shown in figure. Explain the operation of the encoder & hence obtain the output of the encoder for the message input $(d_1 d_2 d_3 d_4 d_5 = 10110)$

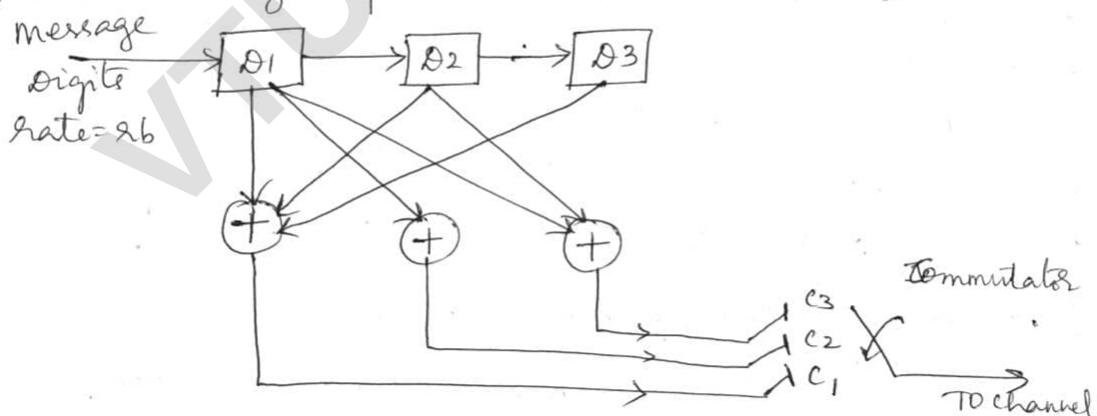


fig: $(3, 1, 3)$ Convolutional encoder

Solⁿ Let the Shift register be cleared initially. The first data bit 'd₁' is entered into the first flip-flop labelled D₁. The commutator samples modulo-2 adders output C₁, C₂ & C₃. Thus single message bit yields three output bits. The next message bit 'd₂' now enters D₁ while the contents of D₁ which was d₁ is shifted into D₂. Then again commutator samples the three adder outputs. This process is repeated till the last message digit d₅ is shifted into the flipflop D₃. In order to move d₅ into D₂ & then into D₃, it is assumed that '0's are fed at the input.

For the i/p message bits (d₁d₂d₃d₄d₅ = 10110).

at time t=0 shift register contents are zero i.e., D₁D₂D₃.

	d ₁ D ₁	D ₂	D ₃	C ₁	C ₂	C ₃
t = 0	0	0	0			
t = t ₁	1	0	0	1	1	1
t = t ₂	0	1	0	1	0	1
t = t ₃	1	0	1	0	1	1
t = t ₄	1	1	0	0	1	0
t = t ₅	0	1	1	0	0	1
t = t ₆	0	0	1	1	0	0
t = t ₆	0	0	0	0	0	0

$$C_1 = D_1 + D_2 + D_3$$

$$C_2 = D_1$$

$$C_3 = D_1 + D_2$$

operation performed
till d₅ reaches D₃
Shift register

In general (n, k, m) convolutional encoder has
 n = number of outputs $\rightarrow$ no. of modulo-2 adders
 k = number of input bits entering at any time
 m = number of stages of shift register or
 number of flip-flops.

L = number of bits in the message sequence or
 length of the message bits

Constraint length = $m \times n$ digits

Rate efficiency = k/n .

Encoding of convolutional codes using Time-Domain Approach.

① Consider $(2, 1, 3)$ convolutional encoder
 with message bits $\rightarrow 10111$. Find the output
 sequence.

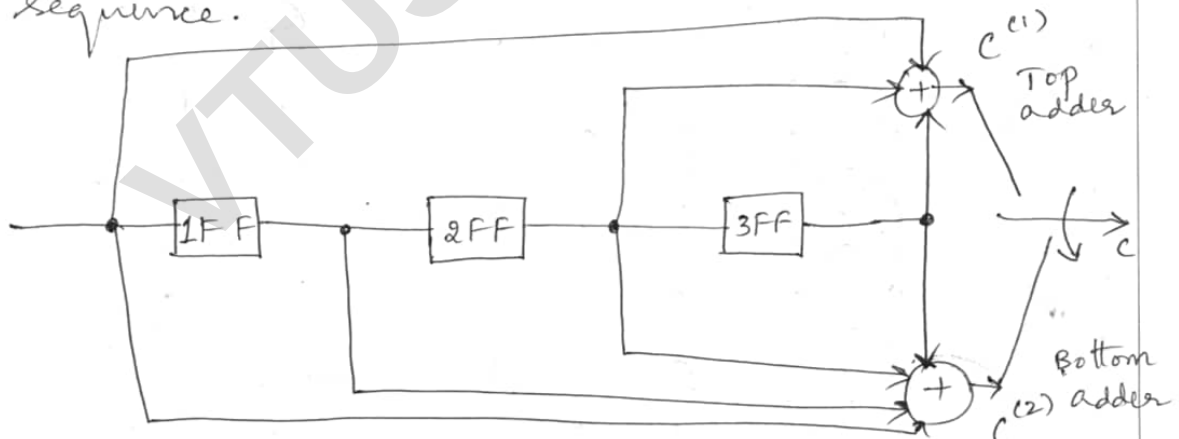


fig: Convolutional encoder $(n, k, m = 2, 1, 3)$

The time-domain behaviour of a binary convolutional encoder is defined in terms of a set of "n-impulse responses".

∴ We need to impulse responses to characterize its behaviour in time domain.

In general, we require "n" number of "impulse response". Let the sequences denote the impulse responses also called "generator sequences" of the input-output path through "n"-modulo 2 adders.

Encoder has 2 modulo 2-adders named as top-adder & bottom adder, hence there will be two generator sequences. Let $(d_1 d_2 \dots d_L)$ represent the input message sequence that enters into the encoder, one bit at a time starting with d_1 . Then the encoder generates output sequences denoted by $c^{(1)}$ & $c^{(2)}$. defined by convolutional sum as

$$c^{(1)} = [d] * g^{(1)}$$

$$c^{(2)} = [d] * g^{(2)}$$

Matrix method: W.K.T $C = D \cdot G$

given that encoder has $D = 10111$

The generator sequence is written with respect to the number of modulo 2 adders. $\therefore$ we have two generator sequences. The matrix can be written with number of rows = no. of bits in message sequence = L rows. & no. of columns = $n(L+m)$

$\therefore$ Such matrix of order $L \times [n(L+m)]$ is called Generator matrix of convolutional encoder.

$\therefore$ The generator matrix has $L=5$ rows & $n(L+m) = 2(5+3) = 16$ columns is given by

$$g^{(1)} = \underline{1011} \text{ \& } g^{(2)} = \underline{1111}$$

$$\therefore G = \begin{bmatrix} 11 & 01 & 11 & 11 & 00 & 00 & 00 & 00 \\ 00 & 11 & 01 & 11 & 11 & 00 & 00 & 00 \\ 00 & 00 & 11 & 01 & 11 & 11 & 00 & 00 \\ 00 & 00 & 00 & 11 & 01 & 11 & 11 & 00 \\ 00 & 00 & 00 & 00 & 11 & 01 & 11 & 11 \end{bmatrix}$$

The encoder output $C = D.G$

$$C = [10111] \begin{bmatrix} 11 & 01 & 11 & 11 & 00 & 00 & 00 & 00 \\ 00 & 11 & 01 & 11 & 11 & 00 & 00 & 00 \\ 00 & 00 & 11 & 01 & 11 & 11 & 00 & 00 \\ 00 & 00 & 00 & 11 & 01 & 11 & 11 & 00 \\ 00 & 00 & 00 & 00 & 11 & 01 & 11 & 11 \end{bmatrix}$$

$$C = [11 \ 01 \ 00 \ 01 \ 01 \ 01 \ 00 \ 11]$$

Encoding of convolutional codes using Transform Domain Approach.

(P1) Obtain the output of convolutional encoder of below figure using transform domain approach.

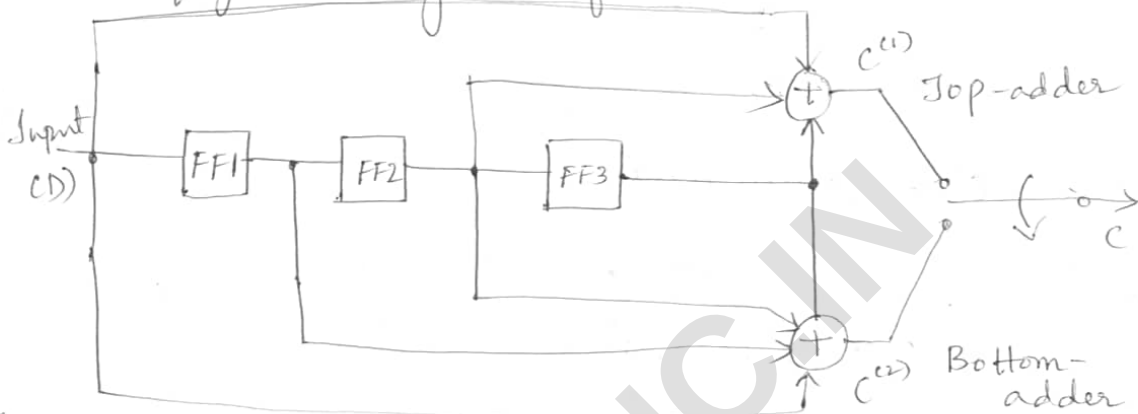


figure (1)

Sol:

The generator sequence for top adder of fig (1) is $g^{(1)} = 1011$, then generator polynomial corresponding to top adder is

$$g^{(1)}(x) = 1 + x^2 + x^3 \quad (1011)$$

Then, the generator polynomial corresponding to bottom adder of $g^{(2)} = 1111$ is

$$g^{(2)}(x) = 1 + x + x^2 + x^3$$

The top adder output polynomial is

$$c^{(1)}(x) = d(x) \cdot g^{(1)}(x)$$

We have $d = 10111 \therefore d(x) = 1 + x^2 + x^3 + x^4$

$$c = D \cdot G$$

$$\therefore C^{(1)}(x) = (1 + x^2 + x^3 + x^4)(1 + x^2 + x^3)$$

$$= 1 + \underline{x^2} + \underline{x^3} + \underline{x^4} + \underline{x^2} + \underline{x^4} + \underline{x^5} + \underline{x^6} + \underline{x^3} + \underline{x^5} + \underline{x^6} + \underline{x^7}$$

$$\therefore C^{(1)}(x) = 1 + x^7$$

$$\therefore C^1 = [10000001]$$

The bottom adder output polynomial,

$$C^2(x) = d(x) \cdot g^2(x)$$

$$\therefore C^2(x) = (1 + x^2 + x^3 + x^4)(1 + x + x^2 + x^3)$$

$$= 1 + \underline{x^2} + \underline{x^3} + \underline{x^4} + \underline{x} + \underline{x^3} + \underline{x^4} + \underline{x^5} + \underline{x^2} + \underline{x^4} + \underline{x^5} + \underline{x^6} + \underline{x^3} + \underline{x^5} + \underline{x^6} + \underline{x^7}$$

$$C^2(x) = 1 + x + x^3 + x^4 + x^5 + x^7$$

$$C^2 = [11011101]$$

The final encoder output is given as for 'n' number of modulo-2 adders = 2

$$C(x) = C^1(x^2) + x C^2(x^2)$$

$$\therefore C^1(x) = 1 + x^7 ; C^1(x^2) = 1 + x^{14}$$

$$C^2(x) = 1 + x + x^3 + x^4 + x^5 + x^7$$

$$C^2(x^2) = 1 + x^2 + x^6 + x^8 + x^{10} + x^{14}$$

$$\therefore c(x) = 1 + x^{14} + x(1 + x^2 + x^6 + x^8 + x^{10} + x^{14})$$

$$c(x) = 1 + x^{14} + x + x^3 + x^7 + x^9 + x^{11} + x^{15}$$

$$c(x) = 1 + x + x^3 + x^7 + x^9 + x^{11} + x^{14} + x^{15}$$

$$C = [11 \ 01 \ 00 \ 01 \ 01 \ 01 \ 00 \ 11]$$

$$\underline{\underline{C}} = [\underline{1} \ \underline{0} \ \underline{0} \ \underline{0} \ \underline{0} \ \underline{0} \ \underline{0} \ \underline{1}] \quad C^2 = [\underline{1} \ \underline{1} \ \underline{0} \ \underline{1} \ \underline{1} \ \underline{1} \ \underline{0} \ \underline{1}]$$

$$C = [\underline{1} \ \underline{1} \ \underline{0} \ \underline{1} \ \underline{0} \ \underline{0} \ \underline{0} \ \underline{1} \ \underline{0} \ \underline{1} \ \underline{0} \ \underline{1} \ \underline{0} \ \underline{0} \ \underline{1} \ \underline{1}]$$

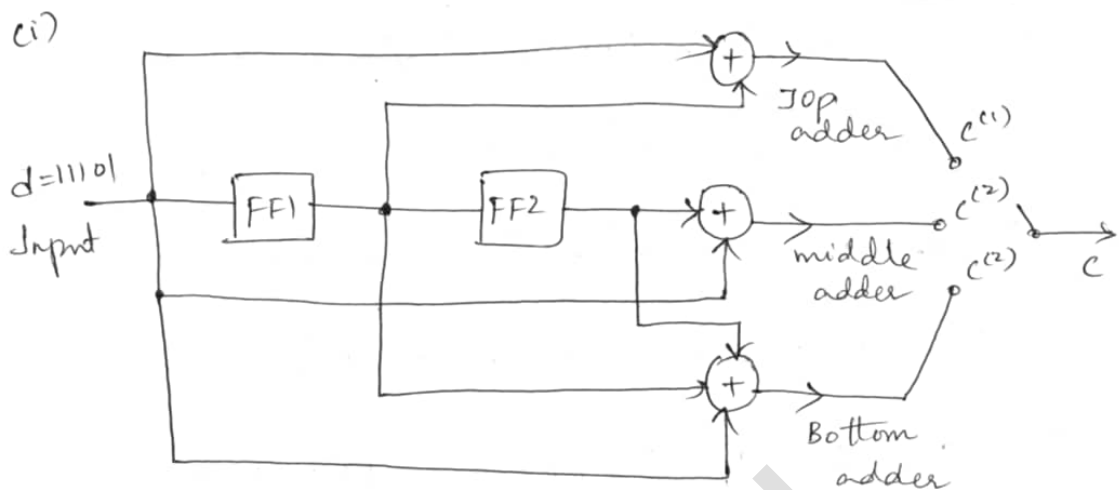
Which is same as that of time-domain approach.

* * P Consider (3, 1, 2) convolutional code with $g^{(1)} = 110$, $g^{(2)} = 101$ & $g^{(3)} = 111$. (a) Draw the encoder block diagram (b) Find the generator matrix (c) Find the code word corresponding to the information sequence (11101) using time domain & transform domain approach.

Solⁿ: Encoder has (3, 1, 2) with $n=3$, $k=1$ & $m=2$

$\therefore$ Encoder has $n=3 \Rightarrow 3 \text{ modulo-2 adders} = 3 \text{ outputs}$
 $m=2 \Rightarrow 2 \text{ Flip flops.}$

$k=1 \Rightarrow 1 \text{ bit at a time will be entered into the flip flop.}$



(ii) Generator matrix has L rows i.e., $5 = L$ &
 $n(L+m) = 3(5+2) = 21$ columns.

$$\therefore G = \begin{bmatrix} 111 & 101 & 011 & 000 & 000 & 000 & 000 \\ 000 & 111 & 101 & 011 & 000 & 000 & 000 \\ 000 & 000 & 111 & 101 & 011 & 000 & 000 \\ 000 & 000 & 000 & 111 & 101 & 011 & 000 \\ 000 & 000 & 000 & 000 & 111 & 101 & 011 \end{bmatrix}$$

$$\begin{aligned} g^{(1)} &= \begin{bmatrix} 111 \\ 101 \\ 011 \end{bmatrix} \\ g^{(2)} &= \begin{bmatrix} 101 \\ 011 \\ 000 \end{bmatrix} \\ g^{(3)} &= \begin{bmatrix} 011 \\ 000 \\ 000 \end{bmatrix} \end{aligned}$$

(iii) The output of the encoder using time-domain approach. $[c] = [d][G]$

$$[c] = [11101] \begin{bmatrix} 111 & 101 & 011 & 000 & 000 & 000 & 000 \\ 000 & 111 & 101 & 011 & 000 & 000 & 000 \\ 000 & 000 & 111 & 101 & 011 & 000 & 000 \\ 000 & 000 & 000 & 111 & 101 & 011 & 000 \\ 000 & 000 & 000 & 000 & 111 & 101 & 011 \end{bmatrix}$$

$$[c] = [111 \ 010 \ 001 \ 110 \ 100 \ 101 \ 011]$$

Using transform domain approach, the message polynomial $d = [11101] = 1 + x + x^2 + x^4$

$$g^{(1)} = 110 \rightarrow 1 + x$$

$$g^{(2)} = 101 \rightarrow 1 + x^2$$

$$g^{(3)} = 111 \rightarrow 1 + x + x^2$$

The output polynomial of the first tap adder is

$$c(x) = d(x)g^{(1)}(x)$$

$$c^1(x) = 1 + x + x^2 + x^4 (1 + x)$$

$$= 1 + \underline{x} + \underline{x^2} + x^4 + \underline{x} + \underline{x^2} + x^5 + x^5$$

$$= 1 + x^3 + x^4 + x^5$$

$$c^1(x) = 1 + x^3 + x^4 + x^5$$

$$c^1 = [1001110]$$

$$c^2(x) = d(x)g^{(2)}(x)$$

$$= 1 + x + x^2 + x^4 (1 + x^2)$$

$$= 1 + x + \underline{x^2} + \underline{x^4} + \underline{x^2} + x^3 + \underline{x^4} + x^6$$

$$c^2(x) = 1 + x + x^3 + x^6$$

$$c^2 = [1101001]$$

$$c^3(x) = d(x)g^{(3)}(x)$$

$$= 1 + x + x^2 + x^4 (1 + x + x^2)$$

$$C^3(x) = 1 + x + x^2 + x^3 + x^4 + x^5 + x^6$$

$$= 1 + x^2 + x^5 + x^6$$

$$C^3 = [1010011]$$

The output of encoder is for $n=3$

$$C(x) = C^1(x^3) + x C^2(x^3) + x^2 C^3(x^3)$$

$$C^1 = 1001110$$

$$C^2 = 1101001$$

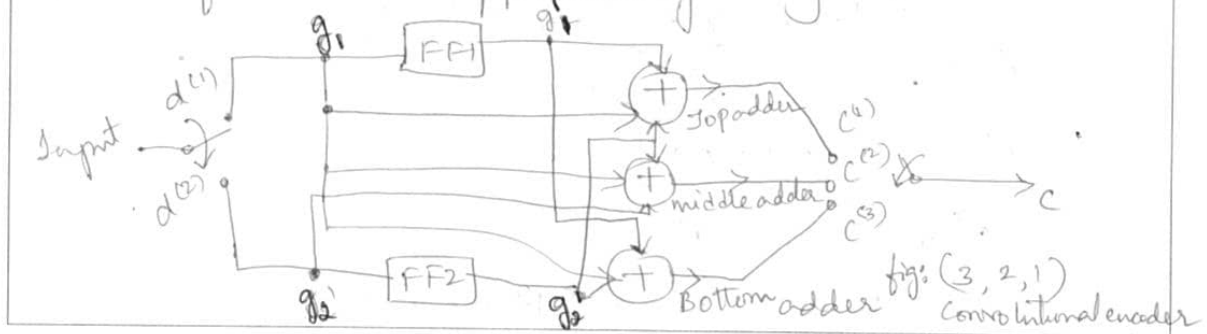
$$C^3 = 1010011$$

$$\therefore C = [111010001110100101011]$$

Hence both transfer & time domain of pare same.

(P2) For $(3, 2, 1)$ encoder shown in fig below. Find code-word 'C' for input sequences of $d^{(1)} = 101$ & $d^{(2)} = 110$ using

- Time domain approach (using generator matrix)
- Transfer domain approach by using matrix.



Solⁿ: (a) The encoder consists of $m=1$ stage shift registers with $n=3$, modulo 2 adders & 2 commutators, one at input & other at o/p. The number of input bits entering through input commutator is $k=2$.

$$\text{for } k=2, \quad d^{(1)} = 101 \\ d^{(2)} = 110$$

There are 2 segments in the input commutator & 3 in output commutator $\therefore i=1, 2$ & $j=1, 2, 3$

$g_1^1 = 11$ Input & o/p of top flip flop to top address connected.

$g_1^2 = 10$ Input connected, o/p not connected for middle address from flip flop 1

$g_1^3 = 11$ both i/p & o/p of flip flop 1 connected to bottom address

$g_2^1 = 01$ i/p of FF2 not connected & o/p connected to top address

$g_2^2 = 11$ both i/p & o/p of FF2 connected to middle address

$g_2^3 = 00$ both i/p & o/p of FF2 not connected to bottom address.

The generator sequence of encoder

$$g_1 = 111, 101$$

$$g_2 = 010, 110$$

The generator matrix will have $L+L-2L$ number of rows
 $\stackrel{\text{6 rows}}{=}$ $n(L+m)$ number of columns, $L=3, n=3, m=1$
 $n(L+m) = 3(3+1) = 12$ columns.

$\therefore G = 6 \times 12 = 6 \text{ rows} \& 12 \text{ columns.}$

$$G = \begin{bmatrix} 111 & 101 & 000 & 000 \\ 010 & 110 & 000 & 000 \\ 000 & 111 & 101 & 000 \\ 000 & 010 & 110 & 000 \\ 000 & 000 & 111 & 101 \\ 000 & 000 & 010 & 110 \end{bmatrix} \begin{matrix} \rightarrow 1^{\text{st}} g_1 \\ \rightarrow 2^{\text{nd}} g_2 \end{matrix}$$

The op of convolutional encoder

$$[C] = [D][G]$$

$$\therefore D = 110110$$

$$d_1 = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}$$

$$d_2 = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}$$

$$\therefore d = \underline{110110}$$

$$[G] = [110110] \begin{bmatrix} 111 & 101 & 000 & 000 \\ 010 & 110 & 000 & 000 \\ 000 & 111 & 101 & 000 \\ 000 & 010 & 110 & 000 \\ 000 & 000 & 111 & 101 \\ 000 & 000 & 010 & 110 \end{bmatrix}$$

$$[C] = [101 \ 001 \ 001 \ 101]$$

$\underline{=}$

(b) The generator polynomial corresponding to generator sequences are

$$\begin{array}{l|l} g_1^1 = 11 \rightarrow 1+x = g_1^1(x) & g_2^1 = 01 \rightarrow x = g_2^1(x) \\ g_1^2 = 10 \rightarrow 1 = g_1^2(x) & g_2^2 = 11 \rightarrow 1+x = g_2^2(x) \\ g_1^3 = 11 \rightarrow 1+x = g_1^3(x) & g_2^3 = 00 \rightarrow 0 = g_2^3(x) \end{array}$$

$\therefore$ The generator polynomial matrix also called "transfer function matrix" is given by

$$G(x) = \begin{bmatrix} g_1^1(x) & g_1^2(x) & g_1^3(x) \\ g_2^1(x) & g_2^2(x) & g_2^3(x) \end{bmatrix}$$

$$G(x) = \begin{bmatrix} 1+x & 1 & 1+x \\ x & 1+x & 0 \end{bmatrix}$$

$$\left. \begin{array}{l} d^1 = 101 \rightarrow 1+x^2 = d^1(x) \\ d^2 = 110 \rightarrow 1+x = d^2(x) \end{array} \right\} \text{message sequence polynomial.}$$

$\therefore$ The code polynomial $C(x) = D(x)G(x)$

$$C(x) = (1+x^2)(1+x) \begin{bmatrix} 1+x & 1 & 1+x \\ x & 1+x & 0 \end{bmatrix}$$

$$= [(1+x^2)(1+x) + (1+x)x \quad (1+x^2)(1+x)^2 \quad (1+x)(1+x^2) + 0]$$

$$C(x) = [1+x^2+x+x^3+x+x^2 \quad 1+x^2+1+x^2+x \quad 1+x^2+x+x^3]$$

$\downarrow$
 0

$$C(x) = [1+x^3 \quad 0 \quad 1+x+x^2+x^3]$$

$$C^1(x) = 1+x^3$$

$$C^2(x) = 0$$

$$C^3(x) = 1+x+x^2+x^3$$

The ofp code word for $n=3$ is written as

$$C(x) = C^1(x^3) + x C^2(x^3) + x^2 C^3(x^3)$$

$$= 1+x^9 + x(0) + x^2(1+x^3+x^6+x^9)$$

$$= 1+x^9 + 0 + x^2 + x^5 + x^8 + x^{11}$$

$$C(x) = 1+x^2+x^5+x^8+x^9+x^{11}$$

$$C = [101 \ 001 \ 001 \ 101]$$

or

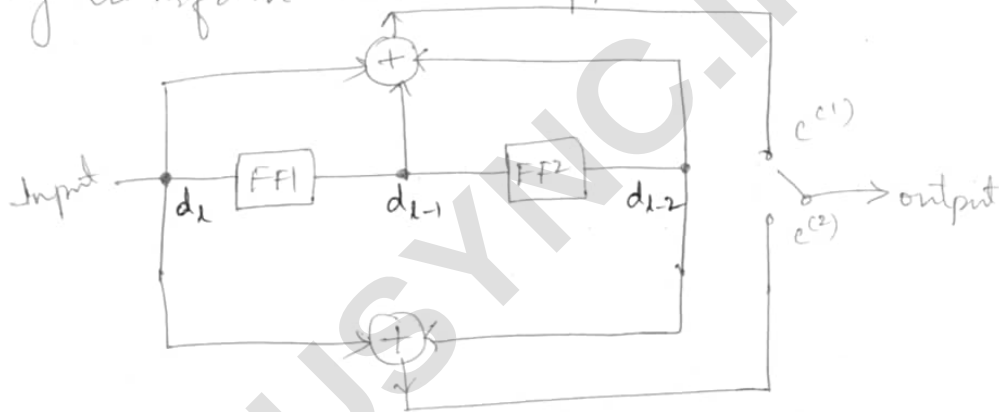
$$\begin{aligned} C^1(x) = 1+x^3 &\rightarrow C^1 = 100 \\ C^2(x) = 0 &\rightarrow C^2 = 0000 \\ C^3(x) = 1+x+x^2+x^3 &= 1111 \end{aligned}$$

$$\therefore C = [101 \ 001 \ 001 \ 101]$$

Which is same as time domain approach.

State diagram and code tree of Convolutional Codes.

(P1) Consider a binary convolutional encoder shown in figure. Draw the state table, state transition table, state diagram & corresponding code tree. Using the code tree, find the encoded sequence for the msg (10111). Verify the output sequence so obtained using transform domain approach.



(2, 1, 2) convolutional encoder.

Sol: 1) State table

There are 2 flip-flops in the shift register & hence there are $2^2 = 4$ states. Let the states be s_0, s_1, s_2 & s_3 .

State	s_0	s_1	s_2	s_3
Binary description	00	10	01	11

$$c^{(1)} = d_k + d_{k-1} + d_{k-2}$$

$$c^{(2)} = d_k + d_{k-2}$$

$\begin{cases} 00 \\ 10 \\ 01 \\ 11 \end{cases}$
 1 bit difference

(2) State transition table.

Present State	Binary Description	Input	Next State	Binary Description	d_k	d_{k-1}	d_{k-2}	c_1	c_2
S_0	00	0	S_0	00	0	0	0	0	0
		1	S_1	10	1	0	0	1	1
* S_1	10	0	S_2	01	0	1	0	1	0
		1	S_3	11	1	1	0	0	1
S_2	01	0	S_0	00	0	0	1	1	1
		1	S_1	10	1	0	1	0	0
S_3	11	0	S_2	01	0	1	1	0	1
		1	S_3	11	1	1	1	1	0

Construction procedure:

Let the flipflop be cleared in first state
 $00 \rightarrow S_0$. With i/p '0', Shift register content is

$S_0 \rightarrow 0 \rightarrow 0 \rightarrow 0 \quad \therefore 00 \rightarrow S_0 \text{ i/p } 0.$
 $\rightarrow 1 \rightarrow 1 \rightarrow 0 \quad \therefore 10 \rightarrow S_1 \text{ i/p } 1.$

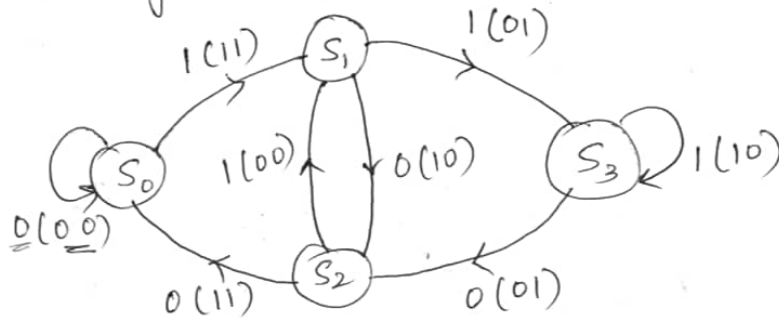
$\therefore$ The i/p & o/p of flf's $d_k d_{k-1} d_{k-2} = 100.$
 & $\therefore$ o/p $c_1 = 00$ & $c_2 = 11.$

Only $S_1 = 10$ (2nd state)*

$\text{i/p } 0 \therefore S_1 \rightarrow 01 \quad \text{i.e., } 01 \rightarrow S_2 \text{ when i/p } 0.$
 $1 \therefore S_1 \rightarrow 11 \quad \text{i.e., } 11 \rightarrow S_3 \text{ when i/p } 1.$

$\therefore$ The i/p & o/p of flf's is $d_k d_{k-1} d_{k-2} = 010$ & $c_1 = 10$
 $c_2 = 01$

(3) State diagram.



In the state diagram, the four states are represented by small circles labelled S_0, S_1, S_2 & S_3 .

From the state transition table, depending on the input 0 or 1, for which state the transition takes place from present state is represented.

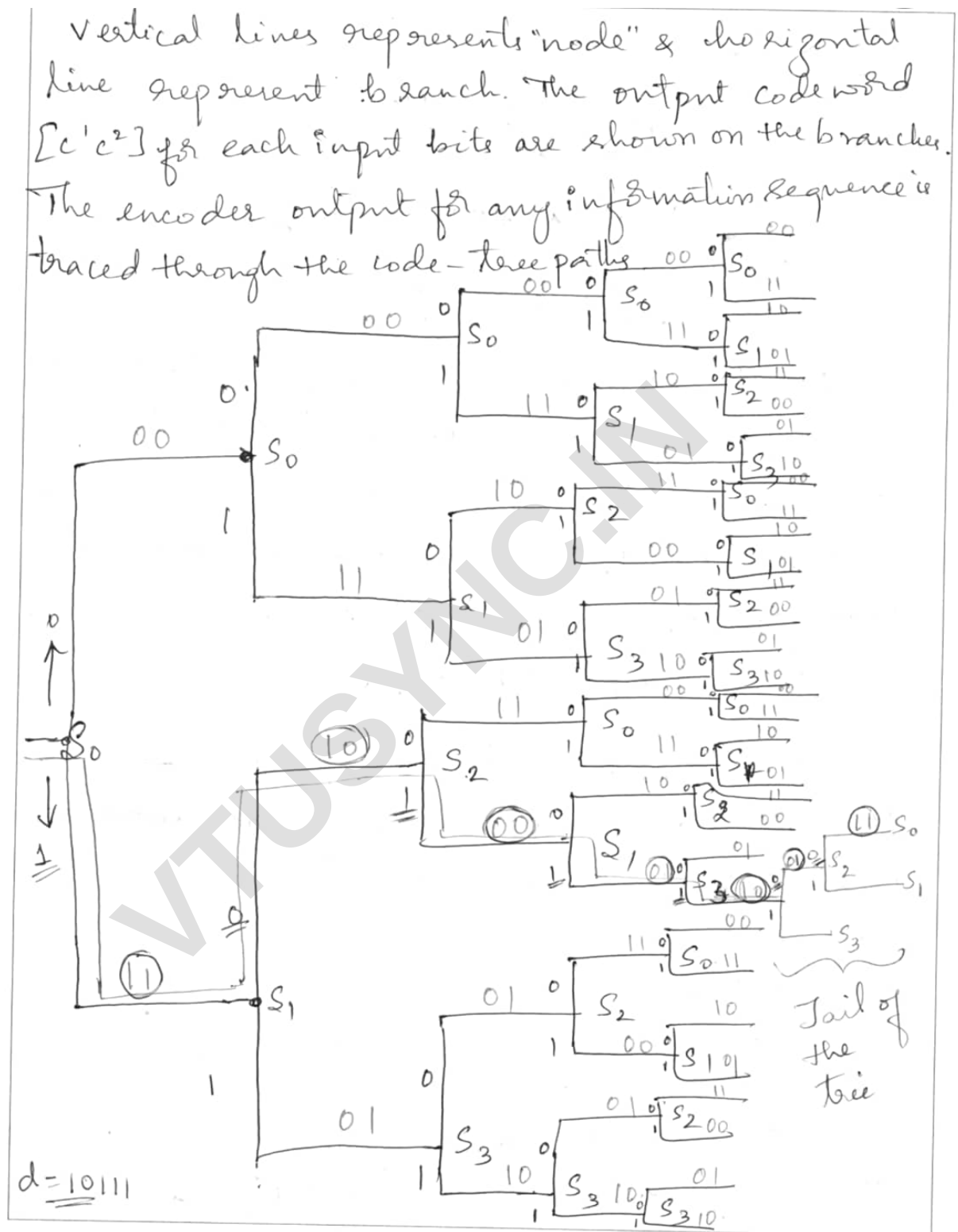
$S_0 \xrightarrow{i/p:0} S_0$ & o/p will be 00. } which is represented
 $S_0 \xrightarrow{i/p:1} S_1$ & o/p will be 11. } as i/p(o/p).

Similarly all the present states are marked with respective input & output, to complete the state diagram.

4) Code Tree:

The state diagram can be re-drawn as "code - tree".

Considers, the i/p '0' follows upward path & the input '1' follows downward path & branch continues.



For any information or message sequence, here 10111, we can traverse through the code-tree in a path shown in green lines.

i.e., when $d = d_5 d_4 d_3 d_2 d_1$, start from $d_5 = 1$, then consider i/p as 1 & follow the path S_1 , next i/p is 0 = d_4 , then follow the path to S_2 (10), next $d_3 = 1$, follow up to S_1 & o/p (00), next $d_2 = 1$, follow up to S_1 & o/p (01), next $d_1 = 1$, follow up to S_3 & o/p (10). $\therefore$ The encoded output will be (11 10 00 01 10). But this is not the complete sequence since the total code length is $n(L+m)$ i.e., $2(5+2) = 14$ bits.

Therefore two more bits are required, hence to complete the code sequence of length nL , the code tree has to be extended, which is called as tail of the tree by appending 'zero' bits in the msg sequence. $\therefore$ i/p $\rightarrow 0$; $S_3 \xrightarrow[0]{01} S_2$ next i/p $\rightarrow 0$; $S_2 \xrightarrow[0]{11} S_0$.

$\therefore$ complete information sequence & the coded o/p is

$$C = [11\ 10\ 00\ 01\ 10\ 01\ 11]$$

Trellis Diagram:

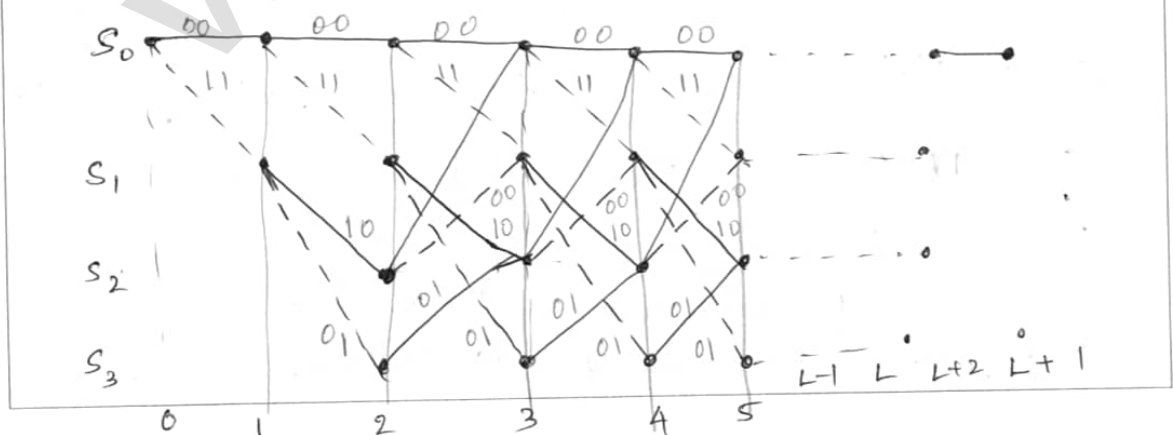
To understand the implications of a convolutional encoder, the trellis diagram is studied.

Also, the code-tree becomes enormously large, when the number of input bits are increased. $\therefore$ Trellis diagram is a compact diagram not as small as state diagram neither large as code tree.

A Trellis is a tree-like structure with ~~re~~merging branches. A trellis is more instructive than a code-tree in that it brings out explicitly the fact that the convolutional encoder is a finite state machine. Trellis diagram is more helpful in decoding the received sequence than either state diagram or code-tree.

By observing the code-tree, after third branch, the nodes are repetitive. It is done using state transition table

The convention used in the construction of a trellis diagram is that a code branch produced by an input "0" is drawn as solid line, while a code branch produced by an input "1" is drawn as broken line.



Verification through Transfer Domain approach.

The generator sequences & the corresponding generator polynomials are given by

$$g^{(1)} = [1 \ 1 \ 1] \Rightarrow 1 + x + x^2 = g^1(x)$$

$$g^{(2)} = [1 \ 0 \ 1] \Rightarrow 1 + x^2 = g^2(x)$$

$$d = 10111 = 1 + x^2 + x^3 + x^4$$

for top adder: $c^1(x) = d(x)g^1(x)$

$$c^1(x) = (1 + x^2 + x^3 + x^4)(1 + x + x^2)$$

$$= 1 + x^2 + x^3 + x^4 + x + x^3 + x^4 + x^5 + x^2 + x^4 + x^6$$

$$c^1(x) = 1 + x + x^4 + x^6$$

$$c^1 = 1100101$$

for bottom adder: $c^2(x) = d(x)g^2(x)$

$$c^2(x) = (1 + x^2 + x^3 + x^4)(1 + x^2)$$

$$= 1 + x^2 + x^3 + x^4 + x^2 + x^4 + x^5 + x^6$$

$$c^2(x) = 1 + x^3 + x^5 + x^6$$

$$c^2 = 1001011$$

o/p of encoder:

$$c(x) = c^1(x^2) + x c^2(x^2)$$

$$= 1 + x^2 + x^8 + x^{12} + x(1 + x^3 + x^5 + x^6)$$

$$c(x) = 1 + x^2 + x^8 + x^{12} + x + x^4 + x^6 + x^{13}$$

$$c(x) = 1 + x + x^2 + x^4 + x^6 + x^8 + x^{12} + x^{13}$$

$$\therefore c = [1 \ 1 \ 0 \ 0 \ 0 \ 1 \ 0 \ 0 \ 1 \ 1] \text{ Same as before.}$$

MODULE 5:

UNIT 5: OTHER TYPES OF ERROR CONTROL CODES

Reed-Solomon (RS) Codes

- These are non-binary BCH codes
- The encoder of RS codes differs from binary encoder & it will operate on multiple bit simultaneously
- The (n, k) RS code takes the groups of m -bit symbols of the incoming binary data stream.
 - * It takes such 'k' number of symbols in one block
 - * Then the encoder adds $(n-k)$ redundant symbols to form the codeword of 'n' symbols.
 - * Thus there are 'n' symbols in the codeword. Each symbol contains 'm' number of bits.
- A t -error correcting q -ary RS codes has the following parameters.

Block length : $n = q - 1 = 2^m - 1$ symbols.

No. of parity check symbols : $n - k = 2t$ symbols.

No. of message size : $k = q - 1 - 2t$ symbols

Minimum distance : $d_{min} = n - k + 1 = 2t + 1$ symbols

Error correcting capability of code : $t = \frac{n - k}{2}$ symbols

- Minimum distance is greater than the number of parity symbols. Hence RS code is called Maximum distance Separable Code (MSD codes)

- For any (n, k) LBC, $d_{min} < (n - k + 1)$, but for RS code $d_{min} = n - k + 1$

- RS codes can be shortened without decreasing d_{min} by changing (n, k) H.e. shortened decoder are also used

Applications of RS codes.

- Storage devices including CD, Blu-ray disc, DVD, barcodes, etc.,
- Wireless & Mobile communications including cellular telephones, microwave links etc.,
- Satellite communications.
- High speed modems
- Digital television / Digital Video Broadcast.

Golay code

- Golay codes are more powerful than hamming code
- (23, 12) golay code is the only known multiple error correcting binary perfect code, that can correct any combination of 3 or fewer errors per 23-bit code word
- Extended golay code is formed by adding one parity bit to the (23, 12) golay code to form (24, 12) extended golay code.
 - From a system perspective extended golay code is more powerful.
- This parity bit increases the minimum hamming distance of the code from seven to eight & yields a code rate " $1/2$ " which is easier to decode in hardware than the (23, 12) golay code.

Shortened cyclic codes.

- For a (n, k) cyclic code, the generator polynomials are divisors of $x^n + 1$. The $x^n + 1$ polynomial has very few divisors. Hence there are few generator polynomials available. This difficulty can be overcome by shortened cyclic codes.
- In shortened cyclic codes, the last 'i' bits out of 'n' bits of the codeword are padded with zeros.
- These 'i' bits are not transmitted. Only $(n-i)$ bits of the codeword are transmitted.
- Thus for (n, k) cyclic code $(n-i, k-i)$ shortened cyclic code is generated.
- The shortened cyclic code is a subset of the cyclic code from which it was derived & hence its minimum distance & error correcting ability is at least same as that of original code.
- The encoding operation ~~of cyclic code~~, syndrome calculation & error correction procedures for shortened cyclic code is same as cyclic codes.

Burst Error Correcting Codes.

- Different codes discussed earlier detect & correct the errors occurring independently at different bit positions.
- Burst errors occur as a cluster of errors. Cyclic & shortened codes can be used to detect these burst errors.
- The burst of length " q " is defined as the vector whose non-zero components are confined to ' q ' consecutive digit positions with non-zero first & last digits.
eg: vector $x = [00101110101000]$ has the burst of length 9.
- A code which is capable of correcting all burst errors of length ' q ' or less is called q -burst error correcting code & is said to have burst error correcting ability q .
- The condition for a q -burst error correcting code (n, k) is that the number of parity bits must be at least $2q$
ie., $n - k \geq 2q$
- Thus we can say that (n, k) burst error correcting code can correct the burst of length upto $\frac{n-k}{2}$. This becomes the upper bound on the burst error correcting capability of (n, k) code i.e., $q \leq \frac{n-k}{2}$.

Burst and Random Error Correcting Codes.

- In most practical systems, errors occur neither independently, at random nor in well defined bursts.

∴ Random error correcting codes & single burst error correcting codes will not be sufficient to correct the mixture of random & burst errors.

- It is better to design codes capable of correcting random errors and/or single error & multiple bursts which has occurred in the channel.

- The most effective method is the Interlacing Technique.

- Given a (n, k) cyclic code, it is possible to construct a $(\lambda n, \lambda k)$ cyclic interlaced code by simply arranging ' λ ' code vectors of the original code into ' λ ' rows of a rectangular array and transmitting them column by column. The resulting code is called an interlaced code with an interlacing degree ' λ '.

Definition of Interlaced Code:

Consider an (n, k) block code. When ' λ ' no. of code words of this code are interlaced then the code becomes $(\lambda n, \lambda k)$. This code is called interlaced code.

Description of BCH Codes

The Bose, Chaudhuri, and Hocquenghem (BCH) codes form a large class of powerful random error-correcting cyclic codes. This class of codes is a remarkable generalization of the Hamming code for multiple-error correction. Consider binary BCH codes, for any positive integer's $m \geq 3$ and $t < 2^{m-1}$, there exists a binary BCH code with the following parameters:

Block length: $n = 2^m - 1$

Number of parity-check digits: $n - k \leq mt$

Minimum distance: $d_{\min} \geq 2t + 1$.

We call this code a t -error-correcting BCH code.

Let α be a primitive element in $GF(2^m)$. The generator polynomial $g(x)$ of the t -error-correcting BCH code of length $2^m - 1$ is the lowest-degree polynomial over $GF(2)$ which has $\alpha, \alpha^2, \alpha^3, \dots, \alpha^{2t}$ as its roots.

$g(\alpha^i) = 0$ for $1 \leq i \leq 2t$ and $g(x)$ has $\alpha, \alpha^2, \dots, \alpha^{2t}$ and their conjugates as all its roots.

The degree of $g(x)$ is at most mt . That is, the number of parity-check digits, $n - k$, of the code is at most equal to mt . If t is small, $n - k$ is exactly equal to mt . Since α is a primitive element, the BCH codes defined are usually called primitive (or narrow-sense) BCH codes.